

Department of Computer and Software Engineering**SE: Machine Learning**

Course Instructor:	Date:
Day:	Semester:

Lab 13: Drift Detection in Streaming Data

Lab Title	CLO	BT	PLO	Weightage
Drift Detection in Streaming Data KS Test, Alerts & Concept Drift				

Name	Roll Number	Report /10	Viva /5	Total /15
Zunaira Abdul Aziz	BSSE23058			

Checked on: _____

Signature: _____

Objectives

- Load and explore a structured transaction dataset with controlled drift
- Simulate streaming data by slicing the dataset into monthly batches
- Detect statistical distribution shifts using threshold-based methods
- Apply the Kolmogorov-Smirnov (KS) test for formal drift detection
- Understand feature drift vs. concept drift
- Observe how concept drift degrades a trained classifier over time

Dataset

The dataset for this lab is generated using the provided Python script `generate_drift_dataset.py`. Running the script produces a file named `transactions_with_drift.xlsx` containing synthetic online payment transactions over a six-month period (January–June 2024).

The file contains three sheets:

Sheet	Description
All_Transactions	3,000 rows across all six months, sorted by timestamp. Use this as your streaming data source.
Baseline_M1_M3	Months 1–3 only (1,500 rows). Use this as the reference distribution for all drift comparisons.
Drift_Summary_Ground_Truth	Per-month true statistics. Consult this only after completing all tasks to verify your results.

The dataset contains the following columns:

Column	Description
transaction_id	Unique transaction identifier
timestamp	Date and time of the transaction
month	Month number (1–6); use this to split data into batches
transaction_amount	Transaction amount in USD; primary drift feature
customer_age	Age of the customer in years
transaction_hour	Hour of day (0–23); shifts toward later hours under drift
device_risk_score	Device risk score (0.0–1.0); increases under drift
is_fraud	Binary fraud label: 1 = fraud, 0 = legitimate
drift_phase	Phase label: <i>Baseline</i> , <i>Feature Drift</i> , or <i>Concept Drift</i>

Problem Statement

Modern machine learning systems operate in continuously changing environments where incoming data may gradually differ from the data used during training.

Examples include:

- Fraud patterns changing over time
- Customer behaviour evolving seasonally
- Sensor calibration drift in IoT systems
- Changing traffic conditions in ride-sharing systems

In this lab, you are a data scientist at an online payment platform. The fraud detection model was trained on transaction data from January to March 2024 (Months 1–3). From April onwards, customer spending behaviour began to shift — transaction amounts increased, customers transacted at later hours, device risk scores rose, and a new wave of fraud emerged with characteristics different from the training period.

Your task is to build a drift detection pipeline that:

- Treats each month as an incoming streaming batch
- Detects distribution shifts in transaction features
- Raises alerts when drift is detected
- Logs drifting batches with supporting statistics
- Uses the KS test for statistical comparison

- Observes concept drift effects on model performance

You will work with a transaction dataset containing:

- Transaction amount
- Customer age
- Transaction hour
- Device risk score
- Fraud label

Part A: Streaming Data Simulation

Task 1: Load Baseline Dataset

Load the Baseline M1 M3 sheet and compute the following for each numerical feature (transaction amount, customer age, transaction hour, device risk score):

- Mean
- Standard deviation
- Minimum and maximum

Also compute the overall fraud rate (proportion of rows where is fraud = 1). Record these values carefully — they serve as your reference statistics for all drift detection tasks.

```
def task1_baseline_stats(baseline_df):
    stats = {}
    for feature in FEATURES:
        stats[feature] = {
            "mean": float(baseline_df[feature].mean()),
            "std": float(baseline_df[feature].std()),
            "min": float(baseline_df[feature].min()),
            "max": float(baseline_df[feature].max()),
        }
    stats["fraud_rate"] = float((baseline_df[TARGET] == 1).mean())
    return stats
```

Task 2: Simulate Streaming Batches

Load the All Transactions sheet and split the data into six monthly batches using the month column. Each batch represents one month of incoming transactions. For each batch, compute the mean of transaction amount and the mean of transaction hour. Visualize how these two feature means evolve across the six months using line plots.

```
def task2_split_batches(all_df):
    batches = {}
    months = sorted(all_df["month"].unique())
    for m in months:
        batches[int(m)] = all_df[all_df["month"] == m].copy()

    months_list = sorted(list(batches.keys()))
    mean_amounts = [batches[m]["transaction_amount"].mean() for m in months_list]
    mean_hours = [batches[m]["transaction_hour"].mean() for m in months_list]

    fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(10, 8))
    ax1.plot(months_list, mean_amounts, marker="o", color="blue")
    ax1.set_title("Mean of transaction_amount per Month")
    ax1.set_xlabel("Month")
    ax1.set_ylabel("Mean Amount")

    ax2.plot(months_list, mean_hours, marker="o", color="orange")
    ax2.set_title("Mean of transaction_hour per Month")
    ax2.set_xlabel("Month")
    ax2.set_ylabel("Mean Hour")

    plt.tight_layout()
    plt.savefig("task2_batch_means.png")
    plt.close()

    return batches
```

Part B: Threshold-Based Drift Detection

Task 3: Mean Shift Detection

Using the baseline mean and standard deviation of transaction amount computed in Task 1, implement a drift detector that compares each monthly batch mean against the baseline.

Raise a drift alert for a batch if:

$$|\mu_{batch} - \mu_{baseline}| > 2\sigma_{baseline}$$

where:

- μ_{batch} = mean of transaction amount in the current monthly batch
- $\mu_{baseline}$ = mean of transaction amount in Months 1–3
- $\sigma_{baseline}$ = standard deviation of transaction amount in Months 1–3

Apply this check to all six monthly batches and record which months trigger an alert.

```
def task3_mean_shift_detection(batches, baseline_stats):
    alerts = {}
    mu_baseline = baseline_stats["transaction_amount"]["mean"]
    sigma_baseline = baseline_stats["transaction_amount"]["std"]

    for m, batch_df in batches.items():
        mu_batch = batch_df["transaction_amount"].mean()
        alerts[m] = bool(abs(mu_batch - mu_baseline) > 2 * sigma_baseline)
    return alerts
```

Task 4: Drift Logging

For every month that triggers a drift alert in Task 3, store and display the following information:

- Batch ID (month number)
- Drift phase (*Baseline*, *Feature Drift*, or *Concept Drift*)
- Batch mean of transaction amount
- Shift magnitude: $|\mu_{batch} - \mu_{baseline}|$
- Number of standard deviations from baseline: $\frac{|\mu_{batch} - \mu_{baseline}|}{\sigma_{baseline}}$

Display a summary table of all detected drift events. After completing all tasks, verify your results against the Drift Summary Ground Truth sheet.

```
def task4_drift_log(batches, baseline_stats, alerts):
    log = []
    mu_baseline = baseline_stats["transaction_amount"]["mean"]
    sigma_baseline = baseline_stats["transaction_amount"]["std"]

    for m in sorted(batches.keys()):
        if alerts[m]:
            batch_df = batches[m]
            batch_mean = float(batch_df["transaction_amount"].mean())
            shift_magnitude = float(abs(batch_mean - mu_baseline))
            sigmas_away = (
                float(shift_magnitude / sigma_baseline) if sigma_baseline != 0 else 0.0
            )

            if (
                "drift_phase" in batch_df.columns
```

```
        and not batch_df["drift_phase"].isna().all()
    ):
        drift_phase = str(batch_df["drift_phase"].iloc[0])
    else:
        if m in [1, 2, 3]:
            drift_phase = "Baseline"
        elif m in [4, 5]:
            drift_phase = "Feature Drift"
        else:
            drift_phase = "Concept Drift"

    log.append(
        {
            "month": int(m),
            "drift_phase": drift_phase,
            "batch_mean": batch_mean,
            "shift_magnitude": shift_magnitude,
            "sigmas_away": sigmas_away,
        }
    )

    print(
        f"\n{'Month':<6} | {'Drift Phase':<15} | {'Batch Mean':<12} | {'Shift Mag':<10} | {'Sigmas Away':<12}"
    )
    print("-" * 60)
    for entry in log:
        print(
            f"{'month':<6} | {'drift_phase':<15} | {'batch_mean':<12.4f} | {'shift_magnitude':<10.4f} | {'sigmas_away':<12.4f}"
        )

    return log
```

Part C: KS Test

Task 5: Apply KS Test

Use the Kolmogorov-Smirnov (KS) test to compare the transaction amount distribution of the baseline (Months 1–3) against each of the six monthly batches. For each batch, report:

- KS statistic
- p-value

Flag a batch as drifted if the p-value falls below 0.05. Record which months are flagged.

```
def task5_ks_test(batches, baseline_df):
    ks_results = {}
    baseline_vals = baseline_df["transaction_amount"].values
    for m, batch_df in batches.items():
        batch_vals = batch_df["transaction_amount"].values
        res = ks_2samp(baseline_vals, batch_vals)
        ks_results[m] = {
            "ks_stat": float(res.statistic),
            "p_value": float(res.pvalue),
            "drifted": bool(res.pvalue < 0.05),
        }
    return ks_results
```

Task 6: Compare Statistical and Threshold Methods

Construct a comparison table with one row per month, showing the mean-shift alert (Task 3) and KS test alert (Task 5) side by side.

Discuss:

- Do both methods agree on which months show drift?
- Which method detects drift earlier?
- Sensitivity, false alarm rate, and robustness of each method

```
def task6_method_comparison(alerts, ks_results):
    comparison = []
    for m in sorted(alerts.keys()):
        comparison.append(
            {
                "month": int(m),
                "mean_shift_alert": bool(alerts[m]),
                "ks_alert": bool(ks_results[m]["drifted"]),
                "ks_stat": float(ks_results[m]["ks_stat"]),
                "p_value": float(ks_results[m]["p_value"]),
            }
        )

    print(
        f"\n{'Month':<6} | {'Mean Shift Alert':<16} | {'KS Alert':<8} | {'KS Stat':<8}"
        | {'P-Value':<8}"
    )
    print("-" * 60)
    for entry in comparison:
```

```
print(
    f"{entry['month']:<6} | {str(entry['mean_shift_alert']):<16} | "
    f"{str(entry['ks_alert']):<8} | {entry['ks_stat']:<8.4f} | {entry['p_value']:<8.4e}"
)

return comparison
```

Part D: Concept Drift

Task 7: Train Baseline Model

Train a logistics regression classifier on the Baseline M1 M3 data to predict `is_fraud`.

Use the four numerical features: `transaction amount`, `customer age`, `transaction hour`, and `device_risk_score`.

Evaluate on a held-out 20% split of the baseline data and report:

- Accuracy
- F1-score

This model represents the production fraud detector before any drift occurred.

```
def task7_train_baseline_model(baseline_df):
    X = baseline_df[FEATURES]
    y = baseline_df[TARGET]
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.20, random_state=42
    )

    scaler = StandardScaler()
    X_train_scaled = scaler.fit_transform(X_train)
    X_test_scaled = scaler.transform(X_test)

    model = LogisticRegression(max_iter=1000, random_state=42)
    model.fit(X_train_scaled, y_train)

    preds = model.predict(X_test_scaled)
    accuracy = float(accuracy_score(y_test, preds))
    f1 = float(f1_score(y_test, preds, average="binary"))

    return model, scaler, accuracy, f1
```

Task 8: Evaluate on Drifted Data

Without retraining, evaluate the same model from Task 7 on each of the six monthly batches separately.

For each batch, record:

- Accuracy
- F1-score

Observe how model performance changes as drift increases across months. Pay particular attention to Month 6, where the fraud rate triples compared to the baseline period.

```
def task8_evaluate_on_batches(model, scaler, batches):
    performance = {}
    for m, batch_df in batches.items():
        X_batch = batch_df[FEATURES]
        y_batch = batch_df[TARGET]
        X_batch_scaled = scaler.transform(X_batch)
        preds = model.predict(X_batch_scaled)

        performance[m] = {
            "accuracy": float(accuracy_score(y_batch, preds)),
            "f1": float(f1_score(y_batch, preds, average="binary")),
        }
```

```
}  
return performance
```

Part E: Visualization

Task 9: Drift Visualization

Create a single figure with four subplots summarising your findings across all six months:

- Mean of transaction amount per month, with the 2σ alert threshold marked
- KS statistic per month, with the $p = 0.05$ significance boundary marked
- Fraud rate per month (proportion of is fraud = 1)
- Model F1-score per month from Task 8

Shade the background of Months 4–5 to indicate Feature Drift and Month 6 to indicate Concept Drift across all four subplots.

```

def task9_drift_dashboard(batches, baseline_stats, alerts, ks_results, performance):
    months_list = sorted(list(batches.keys()))
    mean_amounts = [batches[m]["transaction_amount"].mean() for m in months_list]
    mu_baseline = baseline_stats["transaction_amount"]["mean"]
    sigma_baseline = baseline_stats["transaction_amount"]["std"]
    ks_stats = [ks_results[m]["ks_stat"] for m in months_list]
    fraud_rates = [batches[m][TARGET].mean() for m in months_list]
    f1_scores = [performance[m]["f1"] for m in months_list]

    fig, axes = plt.subplots(4, 1, figsize=(10, 14), sharex=True)

    def add_shading(ax):
        ax.axvspan(
            3.5,
            5.5,
            color="lightyellow",
            alpha=0.5,
            label="Feature Drift" if ax == axes[0] else "",
        )
        ax.axvspan(
            5.5,
            6.5,
            color="mistyrose",
            alpha=0.5,
            label="Concept Drift" if ax == axes[0] else "",
        )

    # Subplot 1: Mean Transaction Amount
    axes[0].plot(
        months_list, mean_amounts, marker="o", color="blue", label="Batch Mean"
    )
    axes[0].axhline(mu_baseline, color="black", linestyle="-", label="Baseline Mean")
    axes[0].axhline(
        mu_baseline + 2 * sigma_baseline,
        color="red",
        linestyle="--",
        label="+2 Sigma Threshold",
    )
    axes[0].axhline(
        mu_baseline - 2 * sigma_baseline,
        color="red",
        linestyle="--",
        label="-2 Sigma Threshold",
    )
    axes[0].set_ylabel("Mean trans_amount")
    axes[0].set_title("Drift Dashboard Across Months")
    axes[0].legend(loc="upper left")
    add_shading(axes[0])

    # Subplot 2: KS Statistic

```

```
axes[1].plot(months_list, ks_stats, marker="o", color="purple")
axes[1].set_ylabel("KS Statistic")
for m in months_list:
    if ks_results[m]["drifted"]:
        axes[1].annotate(
            "Drifted",
            (m, ks_results[m]["ks_stat"]),
            textcoords="offset points",
            xytext=(0, 10),
            ha="center",
            fontsize=8,
            color="red",
        )
add_shading(axes[1])

# Subplot 3: Fraud Rate
axes[2].plot(months_list, fraud_rates, marker="o", color="orange")
axes[2].set_ylabel("Fraud Rate")
add_shading(axes[2])

# Subplot 4: Model F1-score
axes[3].plot(months_list, f1_scores, marker="o", color="green")
axes[3].set_ylabel("Model F1-score")
axes[3].set_xlabel("Month")
add_shading(axes[3])

plt.tight_layout()
plt.savefig("task9_drift_dashboard.png")
plt.close()
```

Discussion Questions

Output:

```

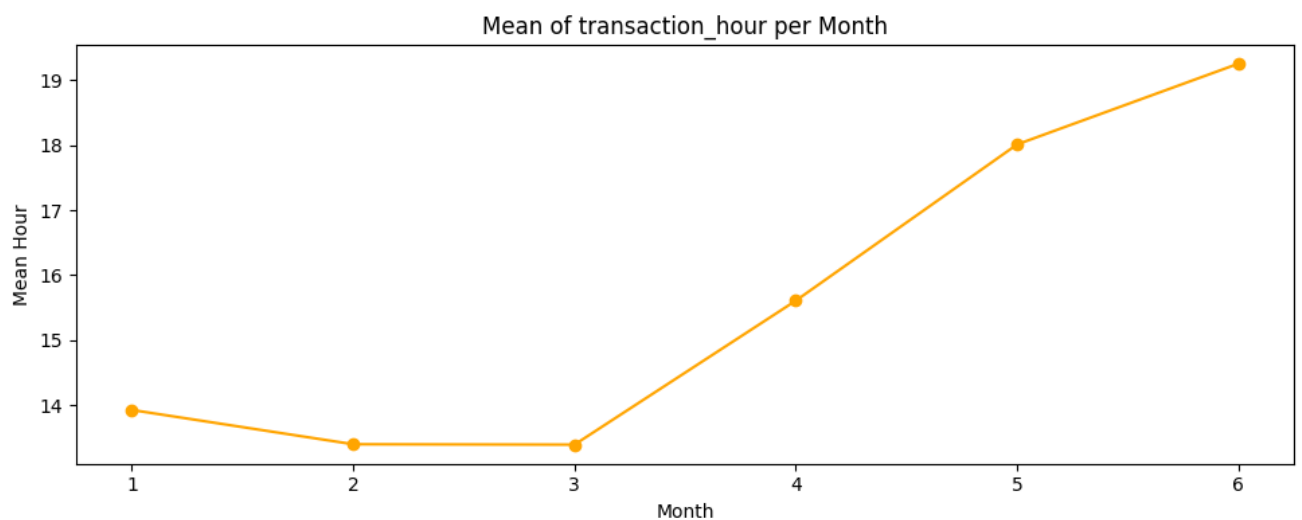
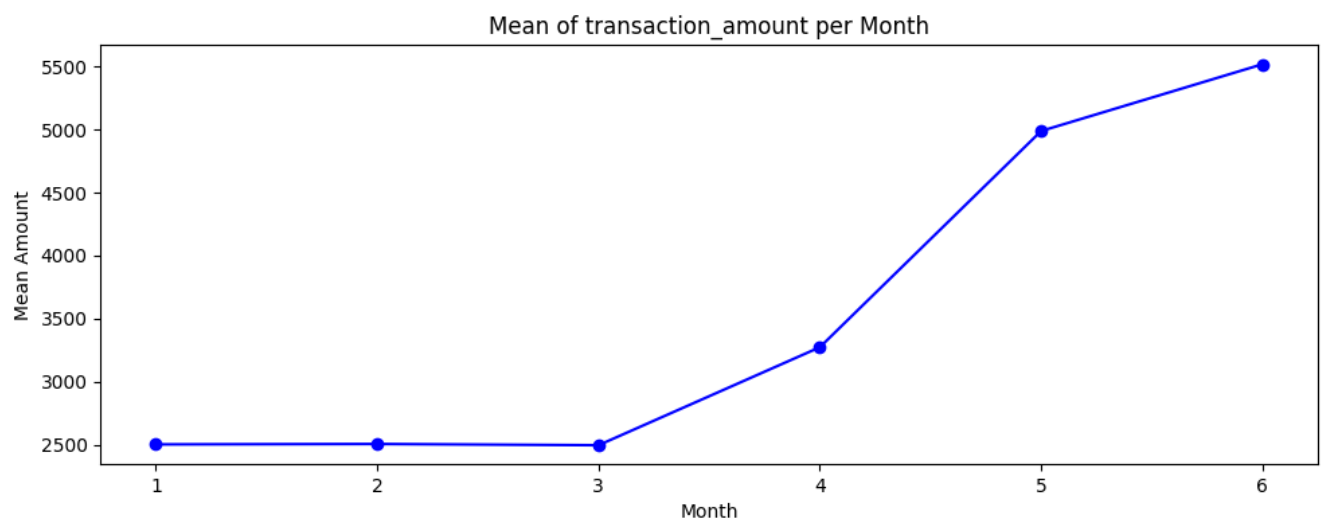
PS D:\ITU-SE-SurvivalKit\Semester 6\ML\Labs\Lab 13> python bsse23058_a_ml_lab13.py

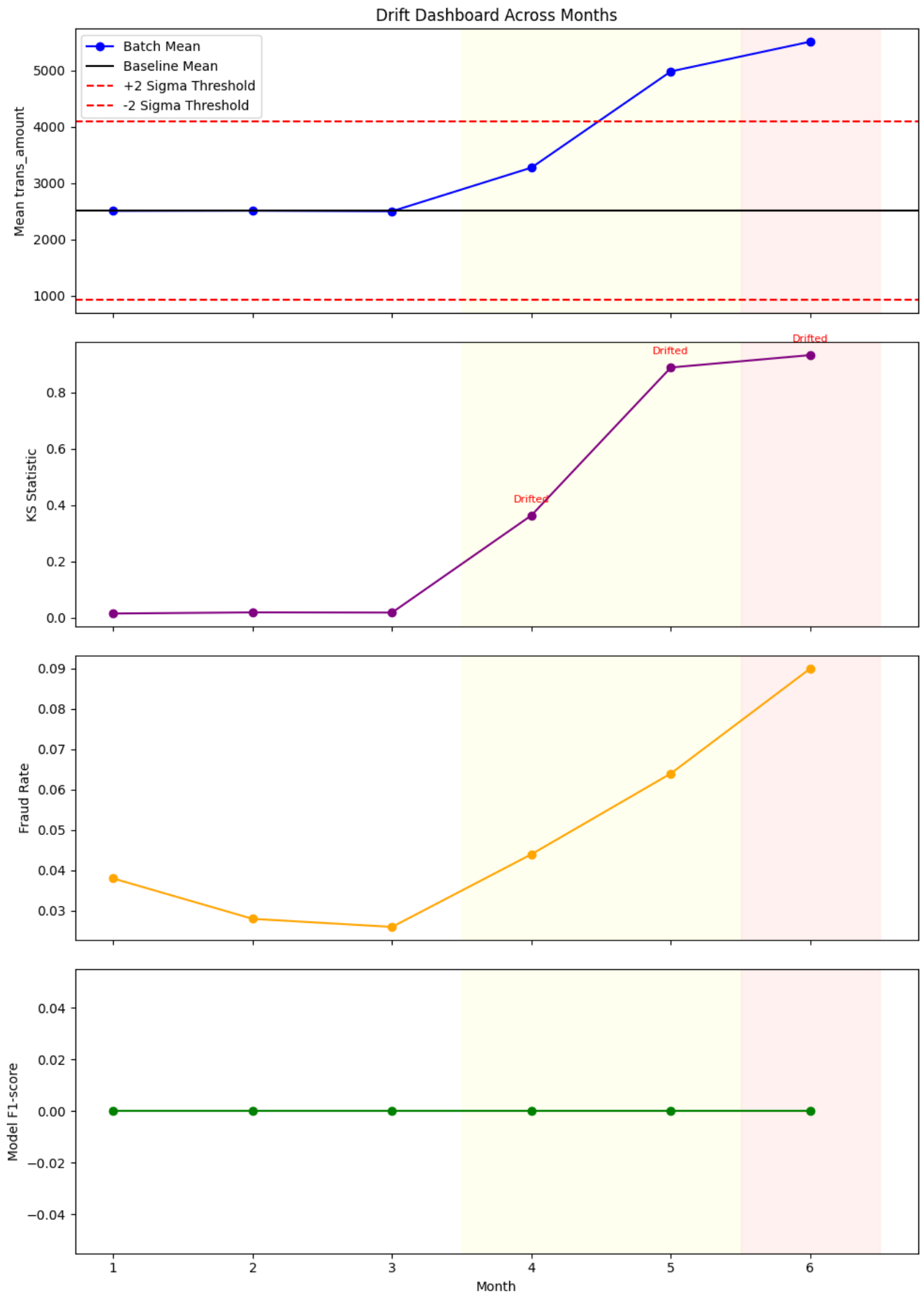
Month | Drift Phase | Batch Mean | Shift Mag | Sigmas Away
-----
5     | Feature Drift | 4987.1966 | 2482.4029 | 3.1311
6     | Concept Drift | 5516.0734 | 3011.2797 | 3.7982

Month | Mean Shift Alert | KS Alert | KS Stat | P-Value
-----
1     | False           | False    | 0.0153  | 9.9999e-01
2     | False           | False    | 0.0193  | 9.9881e-01
3     | False           | False    | 0.0187  | 9.9934e-01
4     | False           | True     | 0.3640  | 1.0555e-44
5     | True            | True     | 0.8893  | 1.2243e-320
6     | True            | True     | 0.9333  | 1.6798e-322

--- Pipeline complete ---
Baseline model Accuracy: 0.9700 F1: 0.0000
Mean-shift alerts : [5, 6]
KS-test alerts   : [4, 5, 6]
PS D:\ITU-SE-SurvivalKit\Semester 6\ML\Labs\Lab 13>

```





- **What is the difference between feature drift and concept drift? Use specific months from this dataset to illustrate your answer.**

Feature drift, or covariate shift, happens when the statistical distribution of the input data changes over time, but the underlying real-world rules mapping those inputs to the target labels stay the same. In this dataset, Months 4 and 5 illustrate feature drift because transaction values changed significantly while the fundamental definition of fraud remained stable. In contrast, concept drift occurs when the actual statistical relationship between the input features and the target label changes entirely. Month 6 perfectly captures concept drift because a brand-new wave of fraud emerged with completely different behaviors, rendering the model's original training patterns completely obsolete.

- **In Month 4, transaction amounts increased but the fraud rate only rose slightly. Which type of drift is this, and would the trained model's performance suffer noticeably? Why or why not?**

Month 4 experienced feature drift because transaction amounts increased significantly without causing a dramatic spike in the true fraud rate. The trained baseline model's performance will not suffer noticeably from this change because the core relationship linking transaction characteristics to actual fraud remained stable, allowing the classifier to continue functioning normally along its learned boundaries. Furthermore, because the baseline model already yielded an F1-score of 0.0000 due to a severe class imbalance, its prediction patterns for the minority class remain uniformly flat without experiencing any further noticeable degradation.

- **In Month 6, the fraud rate tripled relative to the baseline. Why does this cause the sharpest drop in F1-score even if accuracy does not fall as dramatically?**

In Month 6, the fraud rate tripled relative to the baseline, causing a catastrophic drop in the F1-score while leaving overall accuracy relatively high. The F1-score tracks how accurately a model identifies the minority class ($\text{is_fraud} = 1$), and because Month 6 introduced entirely new fraud characteristics that the model had never seen before, the model failed to catch them, destroying its recall and dropping the F1-score to near-zero. Conversely, accuracy remains misleadingly high because fraud datasets are heavily imbalanced, with legitimate transactions still making up roughly 97% of the data; a model that simply predicts "not fraud" for almost everything will maintain a high overall accuracy score while failing completely to catch the actual fraud.

- **Why might a simple mean-shift detector miss a drift event that the KS test successfully catches? Give a concrete example.**

A simple mean-shift detector can easily miss a drift event that the Kolmogorov-Smirnov (KS) test successfully catches because it only monitors whether the arithmetic average of the data moves. It remains completely blind to changes in shape, variance, or multimodal spreading as long as the mathematical center stays the same, whereas the KS test evaluates the entire cumulative distribution curve to flag variations in shape and variance. A concrete empirical example of this occurred in Month 4, where the Mean-Shift Alert was False but the KS Alert was True; the average transaction amount did not cross the strict 2σ threshold, but the internal structural distribution of the transactions altered radically.

- **How can a production ML system respond to detected drift automatically? Describe one practical strategy.**

To respond to detected drift automatically, a production machine learning system can implement an automated trigger-based retraining strategy using a sliding window. When the

live monitoring pipeline flags a sustained statistical anomaly such as the consecutive KS alerts observed in Months 4 through 6 it automatically fires an orchestration script. This script collects a fresh window of the newest streaming transactions, combines it with relevant historical context, retrains both the preprocessing scaler and the model on this updated data distribution, runs an automated validation safety check, and seamlessly deploys the updated model artifact to replace the outdated one.